

Unit I: Introduction to Server-Side Development (4 hours)

- 1.1. Overview of Web -I
- 1.2. HTTP request message, HTTP response Message
- 1.3. Differences between client-side and server-side technologies
- 1.4. Web Clients
- 1.5. Web Server
 - 1.5.1. Role of the server in a web application
 - 1.5.2. Overview of web servers
 - Apache
 - Nginx
 - IIS
- 1.6. Server-side architecture
 - 1.6.1. Monolithic
 - 1.6.2. Microservices
- 1.7. Introduction to backend programming languages (Python, PHP, Java, etc.)
- 1.8. Understanding MVC (Model-View-Controller) architecture

1.1 Overview of Web Technology I

The **Web** is a system of interconnected documents and resources, accessible via the Internet. It enables the exchange of data and services between **web clients** and **web servers**.

What is the Web?

- A collection of servers, clients, and browsers that communicate through **HTTP/HTTPS**.
- Provides the infrastructure to host web applications and services, allowing users to access them via browsers or apps.

Key Components of the Web

1. **Web Clients:**
 - Devices or software that request resources from the server.
 - Examples: Web browsers (Chrome, Firefox), mobile apps.
2. **Web Servers:**
 - Servers that store and serve content, handling client requests and delivering responses.
 - Examples: Apache, Nginx, IIS.
3. **Web Browsers:**
 - Software that allows users to interact with web applications.
 - They interpret and render web content like HTML, CSS, and JavaScript.

How the Web Works

1. **Client Request:** A user enters a URL in the browser or clicks a link to make a request for a resource.
2. **DNS Resolution:** The domain name (e.g., www.nast.edu.np) is translated into an IP address.
3. **Server Response:** The web server processes the request and sends back the requested data (e.g., an HTML file).
4. **Rendering:** The web browser interprets the HTML and displays the content to the user.

Role of URLs and HTTP

- **URLs:** Unique addresses for resources on the web (e.g., <http://www.example.com>).
- **HTTP:** The protocol governing the communication between clients and servers.
 - **HTTP/1.1:** The first widely adopted version of HTTP.
 - **HTTP/2:** Introduced improvements like multiplexing and better resource handling.

Client-side technologies are essential for the structure, presentation, and interaction within a web page, making them the backbone of web development. Let's break down the key technologies.

HTML (Hypertext Markup Language)

- **Purpose:** Defines the structure of a web page by marking up content.
- **Core Components:**
 - **Tags:** HTML uses tags to structure content (e.g., `<div>`, `<header>`, `<footer>`, `<p>`, `<a>`).
 - **Attributes:** Provide additional information (e.g., `class`, `id`, `src`).
- **Example:**

```
<html>
<head><title>My Web Page</title></head>
<body>
  <h1>Welcome to My Website</h1>
  <p>This is an example paragraph. </p>
</body>
</html>
```

CSS (Cascading Style Sheets)

- **Purpose:** Controls the appearance and layout of HTML elements.
- **Core Concepts:**
 - **Selectors:** Target specific elements (e.g., `body`, `.class`, `#id`).
 - **Properties:** Define how elements should be styled (e.g., `color`, `font-family`, `padding`).

- **Responsive Design:** CSS media queries allow for adapting web pages to different screen sizes.
- **Example:**

```
body {  
  font-family: Arial, sans-serif;  
  background-color: #f0f0f0;  
}
```

```
h1 {  
  color: #333;  
  text-align: center;  
}
```

JavaScript (JS)

- **Purpose:** Adds interactivity and dynamic behavior to web pages.
- **Core Concepts:**
 - **DOM Manipulation:** JavaScript interacts with HTML elements to update content and structure dynamically.
 - **Event Handling:** Responds to user interactions such as clicks or keyboard inputs.
 - **AJAX and Fetch:** Asynchronous techniques to load data from the server without refreshing the page.
- **Example:**

```
document.getElementById('myButton').addEventListener('click', function() {  
  alert('Button clicked!');  
});
```

Additional Client-Side Technologies

1. **jQuery:** A popular JavaScript library simplifying DOM manipulation and event handling.
2. **Frameworks/Libraries:**
 - **React, Angular, Vue:** Modern frameworks that enable the development of interactive, single-page web applications.
3. **WebAssembly:** A binary format for running high-performance code (like C++ or Rust) in the browser.

Client-side Rendering (CSR) vs. Server-side Rendering (SSR)

- **CSR:** JavaScript dynamically loads content in the browser (common in SPAs).
- **SSR:** The server renders the HTML before sending it to the client, making the page load faster initially (traditional web apps).

1.2 HTTP Request and Response Messages

HTTP (HyperText Transfer Protocol) defines how messages are formatted and transmitted between clients and servers.

HTTP Request Message

The client (e.g., browser, mobile app) sends an HTTP request message to the server to retrieve or send data.

Structure:

1. **Request Line:**
 - **Method:** Describes the action (e.g., GET, POST).
 - **URL:** The resource being requested.
 - **HTTP Version:** E.g., HTTP/1.1 or HTTP/2.
2. **Headers:** Contains metadata such as:
 - **User-Agent:** Information about the client (e.g., browser).
 - **Accept:** Specifies what kind of response is acceptable (e.g., text/html, application/json).
3. **Body:** (Optional) Contains data sent with the request, typically used with POST or PUT methods.

Example Request:

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
```

HTTP Response Message

After receiving a request, the server sends back an HTTP response message.

Structure:

1. **Status Line:**
 - **HTTP Version:** E.g., HTTP/1.1.
 - **Status Code:** A 3-digit code indicating the result (e.g., 200 OK, 404 Not Found).
2. **Headers:** Contains metadata about the response, such as:
 - **Content-Type:** The type of data being returned (e.g., text/html, application/json).
 - **Date:** The time the response was generated.
3. **Body:** Contains the requested content or data.

Example Response:

HTTP/1.1 200 OK
Content-Type: text/html
Date: Tue, 01 May 2025 10:00:00 GMT

<html>...</html>

HTTP Methods

- **GET**: Fetch data from the server.
- **POST**: Send data to the server.
- **PUT**: Modify data on the server.
- **DELETE**: Remove data from the server.

HTTP Status Codes

- **2xx**: Success (e.g., 200 OK, 201 Created)
- **3xx**: Redirection (e.g., 301 Moved Permanently)
- **4xx**: Client Errors (e.g., 404 Not Found)
- **5xx**: Server Errors (e.g., 500 Internal Server Error)

1.3 Differences Between Client-side and Server-side Technologies

Web applications are built using both **client-side** and **server-side** technologies to deliver dynamic, secure, and interactive experiences.

Client-side Technologies

- Executed in the user's web browser.
- Common technologies: **HTML, CSS, JavaScript**.
- Responsible for:
 - UI presentation
 - Form validation
 - User interaction and animations
- Pros:
 - Fast, responsive interfaces
 - Offloads work from server
- Cons:
 - Cannot directly access server/database
 - Vulnerable to manipulation if not properly validated

Server-side Technologies

- Executed on a web server.
- Common technologies: **PHP, Python, Java, Node.js, Ruby**.
- Responsible for:

- Handling requests
 - Database interactions
 - Business logic
 - Authentication and authorization
- Pros:
 - Centralized control
 - Secure data processing
- Cons:
 - Higher load on server
 - Slower if not optimized

Comparison Table

Feature	Client-side	Server-side
Execution Location	Browser	Web server
Technologies	HTML, CSS, JavaScript	PHP, Python, Java, Node.js
Access to Database	✗	✓
Speed	Fast for UI	Slower for heavy logic
Security	Less secure	More secure

1.4 Web Clients

Web clients are the "consumers" of web applications. They request content or services from a server and present it to users.

Key Responsibilities

- **Request Handling:** Send HTTP/HTTPS requests to web servers.
- **Rendering:** Display web content (HTML, CSS) and run JavaScript.
- **User Interaction:** Allow users to input data and navigate the application.
- **Session Management:** Store cookies, tokens, and local/session storage.

Types of Web Clients

- **Web Browsers:** Standard interface for accessing web applications.
- **Mobile Applications:** Native apps that communicate with backend servers.
- **API Clients:** Tools like Postman or curl for testing server endpoints.
- **Desktop Apps:** Applications built with web technologies (e.g., Electron).

Importance in Architecture

- Web clients are the **entry point** of most user interactions in web systems.
- Their design affects user experience, accessibility, and responsiveness.

1.5 Web Server

1.5.1 Role of the Server in a Web Application

- A web server processes incoming HTTP requests from clients and returns appropriate responses.
- Static content (like HTML files, images, CSS, and JavaScript) is served directly.
- Dynamic content is generated via communication with application servers (e.g., using Python, PHP, Node.js).
- It plays a vital role in:
 - Serving content
 - Enforcing security (SSL/TLS)
 - Caching and compression
 - Load balancing and proxying

1.5.2 Overview of Web Servers

Apache HTTP Server

- Widely used, highly configurable web server.
- Supports CGI, PHP, Python, and many extensions via modules.
- Uses .htaccess files for per-directory configurations.
- Ideal for hosting individual or small business websites.

Nginx

- Designed for high performance and efficient resource usage.
- Commonly used as a reverse proxy and load balancer.
- Handles thousands of simultaneous connections with low memory usage.
- Preferred for large-scale, high-traffic sites.

IIS (Internet Information Services)

- Proprietary web server developed by Microsoft.
- Deeply integrated with ASP.NET and Windows Server.
- GUI-based administration and strong security features.
- Common in enterprise environments and intranet systems.

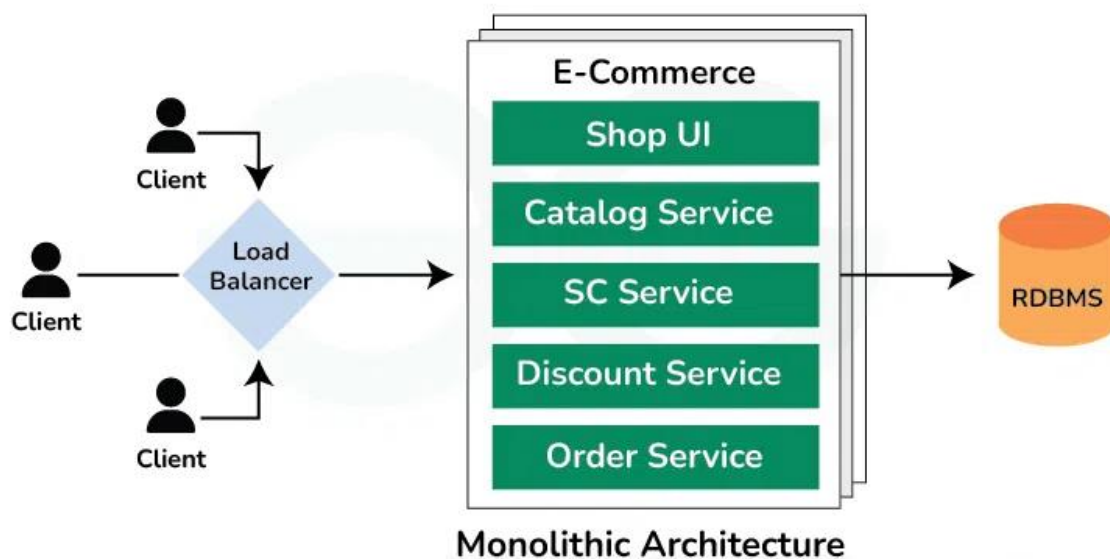
1.6 Server-side Architecture

The structure of the backend of a web application greatly affects how scalable and maintainable it is. Two common server-side architectures are:

1.6.1 Monolithic Architecture

Software is traditionally designed using a monolithic architecture, in which the entire program is constructed as a single, indivisible unit. Every component of the program, including the data access layer, business logic, and user interface, is deployed and integrated tightly together in this design.

- This means that any changes or updates to the application require modifying and redeploying the entire monolith.
- Monolithic architectures are often characterized by their simplicity and ease of development, especially for small to medium-sized applications.
- However, they can become complex and difficult to maintain as the size and complexity of the application grow.



In summary,

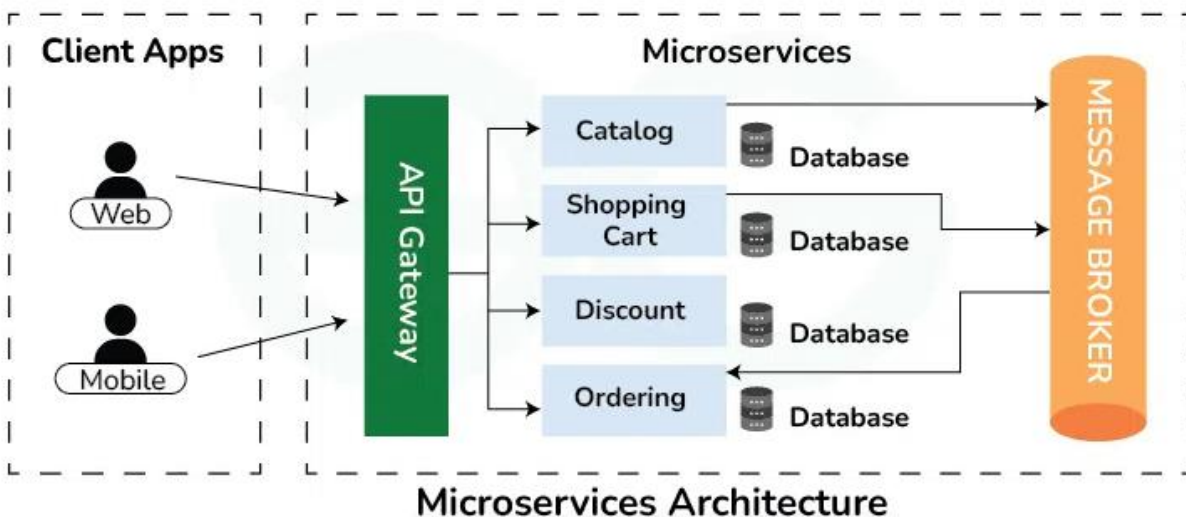
- A monolithic architecture is a **single-tiered** application where all components are combined into one program.
- Includes frontend rendering, backend logic, and data access in a single deployment.
- **Advantages:**
 - Simple to develop and deploy for small teams or projects
 - Easy to debug in early stages
- **Disadvantages:**
 - Difficult to scale specific features
 - Any small change can require full redeployment
 - Tight coupling makes long-term maintenance harder

1.6.2 Microservices Architecture

A microservices architecture results in an application designed as a set of small, independent services. Each one represents a business capability in itself. The services loosely couple with one

another and communicate over the network, typically making use of lightweight protocols such as HTTP or messaging queues.

- Each service is responsible for a single functionality or feature of the application and can be developed, deployed, and scaled independently.
- The Microservice architecture has a significant impact on the relationship between the application and the database.



In summary,

- Microservices architecture splits the application into **smaller, independent services**, each responsible for a specific business function.
- Services communicate via APIs, typically using HTTP/REST, gRPC, or message queues.
- **Advantages:**
 - High scalability and flexibility
 - Independent development and deployment of services
 - Fault isolation – failure in one service doesn't crash the whole system
- **Disadvantages:**
 - More complex to build, test, and manage
 - Requires robust communication and monitoring systems

1.7 Introduction to Backend Programming Languages

Backend development involves programming that happens on the server side. These languages manage business logic, databases, APIs, and integrations.

Python

Python is one of the most popular programming languages today, widely used for web development, data science, machine learning, automation, and more.

Key Features of Python

1. **Readability:** Python's syntax is simple, making it easy to learn and use.
2. **Interpreted:** Python code is executed line by line, rather than compiled, which allows for easier debugging and rapid development.
3. **Dynamically Typed:** You don't need to declare variable types; Python automatically determines the type based on the value.
4. **Cross-Platform:** Python runs on various operating systems like Windows, macOS, and Linux, making it versatile.
5. **Extensive Libraries:** Python has a rich ecosystem of libraries and frameworks for a variety of tasks, from web development to machine learning.

PHP

PHP (Hypertext Preprocessor) is a widely used server-side scripting language designed for creating dynamic and interactive web pages. It is embedded in HTML and executed on the server.

Key Features of PHP

- **Server-Side Scripting:** PHP scripts are executed on the server, generating dynamic HTML content sent to the client's browser.
- **Cross-Platform:** PHP runs on many platforms, making it very versatile (Windows, macOS, Linux).
- **Open Source:** PHP is free to use and has a large, active community for support.
- **Database Integration:** PHP integrates well with databases, especially **MySQL**, making it ideal for building data-driven web applications.

Java

Java is a widely used object-oriented programming language that provides platform independence, security, and robust features, making it suitable for building applications ranging from mobile apps to enterprise-level software.

Key Features of Java

1. **Object-Oriented:** Java is built around the concept of classes and objects, ensuring reusability and scalability of code.
2. **Platform Independent:** Java code is compiled into bytecode, which runs on any system with a Java Virtual Machine (JVM).
3. **Robust:** Java's strong memory management and exception handling ensure reliable applications.
4. **Multithreaded:** Java supports multi-threading, allowing multiple tasks to run simultaneously.

5. **Security:** Java's bytecode verification and sandboxing provide enhanced security.

Other Languages

- **Node.js:** Uses JavaScript on the backend; great for real-time apps.
- **Ruby:** High productivity, used with Ruby on Rails.
- **C# (.NET):** Developed by Microsoft, ideal for Windows environments.
- **Go:** Fast, lightweight, suitable for concurrent services and APIs.

1.8 Understanding MVC (Model-View-Controller) Architecture

MVC is a widely adopted software architectural pattern used in web development. It helps separate concerns, making applications easier to manage.

Model

- Manages the data, logic, and rules of the application.
- Interacts directly with the database or other data sources.
- Sends updates to the view when data changes.

View

- Represents the presentation layer.
- Displays data to the user based on the state of the model.
- Designed to be passive – does not modify data directly.

Controller

- Handles user input (e.g., mouse clicks, form submissions).
- Calls model methods to update data.
- Selects which view to display based on logic.

Advantages of MVC

- Codebase is modular and organized.
- Teams can work simultaneously on different parts.
- MVC frameworks speed up development and follow best practices.

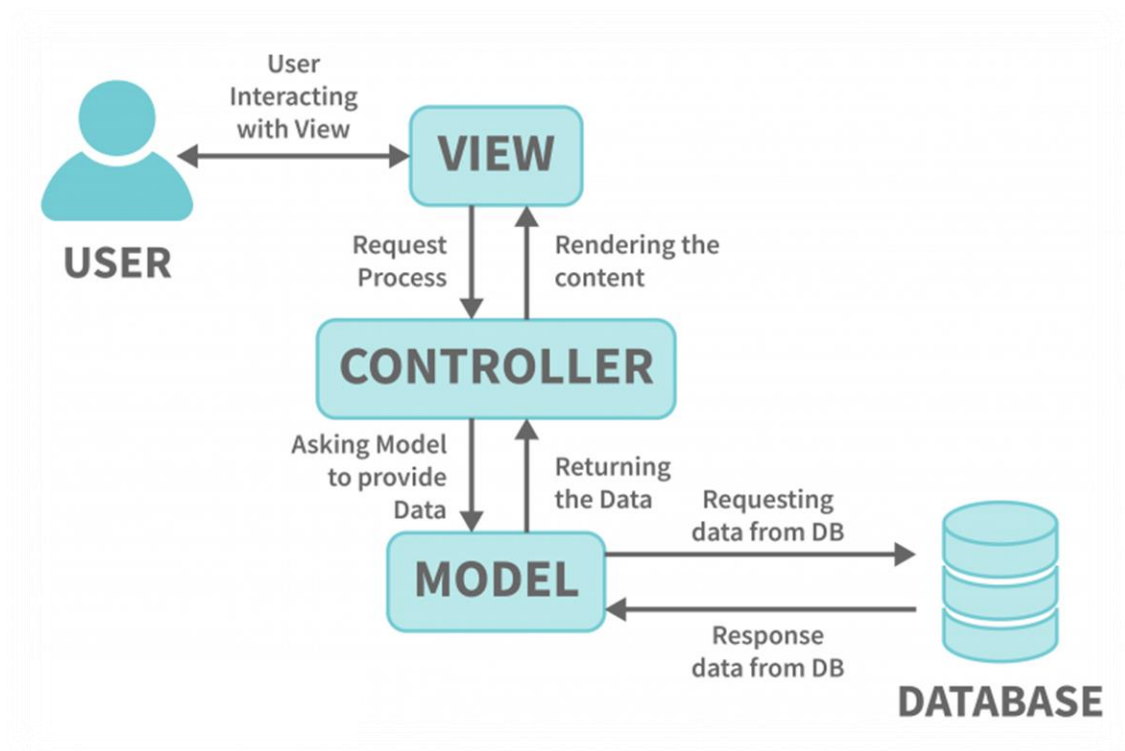


Figure 1 MVC Architecture